INDEX

S.NO	EXPERIMENT NAME	Page No.	DATE	SIGNATURE / REMARKS
1	Basic data types and operators: Create a program that prompts the user for their name and age and prints a personalized message.	1-2		
2	Conditional statements: Create a program that prompts the user for their age and tells them if they can vote in the next election.	3-4		
3	Loops: Create a program that calculates the factorial of a number entered by the user using a loop.	5-6		
4	Lists and arrays: Create a program that prompts the user for a list of numbers and then sorts them in ascending order.	7-8		
5	Strings and string manipulation: Create a program that prompts the user for a string and then prints out the string reversed.	9-10		
6	Functions: Create a program that defines a function to calculate the area of a circle based on the radius entered by the user.	11-12		
7	Classes and objects: Create a program that defines a class to represent a car and then creates an object of that class with specific attributes.	13-14		
8	File input/output: Create a program that reads data from a file and writes it to another file in a different format.	15-16		
9	Regular expressions: Create a program that uses regular expressions to find all instances of a specific pattern in a text file.	17-19		
10	Exception handling: Create a program that prompts the user for two numbers and then divides them, handling any exceptions that may arise.	20-21		
11	GUI programming: Create a program that uses a graphical user interface (GUI) to allow the user to perform simple calculations.	22-25		
12	Web scraping: Create a program that uses a web scraping library to extract data from a website and then stores it in a database.	26-29		
13	Data visualization: Create a program that reads data from a file and then creates a visualization of that data using a data visualization library.	30-31		
14	Machine learning: Create a program that uses a machine learning library to classify images based on their content.	32-34		
15	Networking: Create a program that uses a networking library to communicate with a server and retrieve data from it.	35-37		

EXPERIMENT - 1

AIM :

ALGORITHM :

CODE :

```
# Prompt the user for their name
name = input("Enter your name: ")

# Prompt the user for their age
age = int(input("Enter your age: "))

# Print a personalized message
print(f"Hello, {name}! You are {age} years old.")
```

OUTPUT :

```
>>>
==== RESTART: C:/Users/arya/AppData/Local/Programs/Python/Python38-32/LAB.PY ===
Enter your name:  Radhika
Enter your age: 21
Hello, Radhika! You are 21 years old.
>>>
```

EXPERIMENT - 2

AIM :

ALGORITHM :

CODE :

```
# Prompt the user for their age
age = int(input("Enter your age: "))


# Check voting eligibility
if age >= 18:
    print("You are eligible to vote in the next election! ")
else:
    print(f"Sorry, you need to wait {18 - age} more years to vote.")
```

OUTPUT :

```
Enter your age: 21
You are eligible to vote in the next election!
>>>
```

EXPERIMENT - 3

AIM :

ALGORITHM :

CODE :

```
# Prompt the user for a number
num = int(input("Enter a number: "))


# Initialize factorial to 1
factorial = 1
# Calculate factorial using a loop
if num < 0:
    print("Factorial is not defined for negative numbers.")
elif num == 0 or num == 1:
    print(f"The factorial of {num} is 1.")
else:
    for i in range(1, num + 1):
        factorial *= i
    print(f"The factorial of {num} is {factorial}.")
```

OUTPUT :

```
Enter a number: 7
The factorial of 7 is 5040.
>>>
```

EXPERIMENT - 4

AIM :

ALGORITHM :

CODE :

```
# Prompt the user for a list of numbers
numbers = list(map(int, input("Enter numbers separated by spaces: ").split()))


# Implementing Bubble Sort algorithm
n = len(numbers)
for i in range(n):
    for j in range(0, n-i-1):
        if numbers[j] > numbers[j+1]:
            numbers[j], numbers[j+1] = numbers[j+1], numbers[j]  # Swap


# Display the sorted list
print("Sorted numbers in ascending order:", numbers)
```

OUTPUT :

```
Enter numbers separated by spaces: 7 1 5 3 10 9 11
Sorted numbers in ascending order: [1, 3, 5, 7, 9, 10, 11]
>>>
```

EXPERIMENT - 5

AIM :

ALGORITHM :

CODE :

```
# Prompt the user for a string
user_string = input("Enter a string: ")

# Reverse the string using slicing
reversed_string = user_string[::-1]

# Display the reversed string
print("Reversed string:", reversed_string)
```

OUTPUT :

```
Enter a string: Today Is Tuesday
Reversed string: yadseuT sI yadoT
>>>
```

EXPERIMENT - 6

AIM :

ALGORITHM :

CODE :

```
import math  # Import math module for π (pi)

# Function to calculate the area of a circle
def calculate_area(radius):
    return math.pi * radius ** 2

# Prompt the user for the radius
radius = float(input("Enter the radius of the circle: "))

# Call the function and display the result
area = calculate_area(radius)
print(f"The area of the circle with radius {radius} is: {area:.2f}")
```

OUTPUT :

```
Enter the radius of the circle: 12
The area of the circle with radius 12.0 is: 452.39
>>>
```

EXPERIMENT - 7

AIM :

ALGORITHM :

CODE :

```
# Define the Car class
class Car:
    def __init__(self, brand, model, year, color):
        self.brand = brand
        self.model = model
        self.year = year
        self.color = color

    def display_info(self):
        print(f"Car Details: {self.year} {self.color} {self.brand} {self.model}")

# Create an object of the Car class
my_car = Car("Toyota", "Corolla", 2022, "Blue")

# Call the method to display car details
my_car.display_info()
```

OUTPUT :

```
Car Details: 2022 Blue Toyota Corolla
>>>
```

EXPERIMENT - 8

AIM :

ALGORITHM :

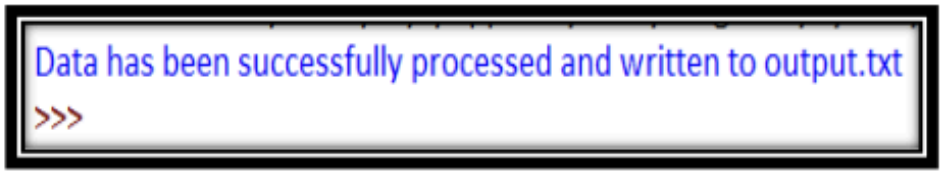
CODE :

```
# Read from input.txt and write to output.txt in uppercase format

# Read data from the input file

with open("input.txt", "r") as infile:

    data = infile.read()

# Process the data (convert to uppercase)

processed_data = data.upper()

# Write the processed data to the output file

with open("output.txt", "w") as outfile:

    outfile.write(processed_data)

print("Data has been successfully processed and written to output.txt")
```

OUTPUT :

- File: input.txt

- Python program run output



- File: output.txt

EXPERIMENT - 9

AIM :

ALGORITHM :

CODE :

```
import re

# Define the pattern to search for (e.g., email addresses)
pattern = r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"

# Read data from the input file
with open("input.txt", "r") as infile:
    data = infile.read()

# Find all occurrences of the pattern
matches = re.findall(pattern, data)

# Write the matches to an output file
with open("output.txt", "w") as outfile:
    for match in matches:
        outfile.write(match + "\n")
print(f"Found {len(matches)} matches. Results saved in output.txt.")
```

OUTPUT :

- File: input.txt

- Python program run output

- File: output.txt

EXPERIMENT - 10

AIM :

ALGORITHM :

CODE :

```
# Function to divide two numbers with exception handling
def safe_division():
    try:
        # Prompt the user for two numbers
        num1 = float(input("Enter the first number: "))
        num2 = float(input("Enter the second number: "))
        # Perform division
        result = num1 / num2
    except ValueError:
        print("Error: Please enter valid numbers.")
    except ZeroDivisionError:
        print("Error: Division by zero is not allowed.")
    else:
        print(f"Result: {num1} ÷ {num2} = {result}")
    finally:
        print("Thank you for using the division calculator!")
# Call the function
safe_division()
```

OUTPUT :

```
Enter the first number: 12
Enter the second number: 3
Result: 12.0 ÷ 3.0 = 4.0
Thank you for using the division calculator!
>>>
```

```
Enter the first number: 2
Enter the second number: 0
Error: Division by zero is not allowed.
Thank you for using the division calculator!
>>>
```

EXPERIMENT - 11

AIM :

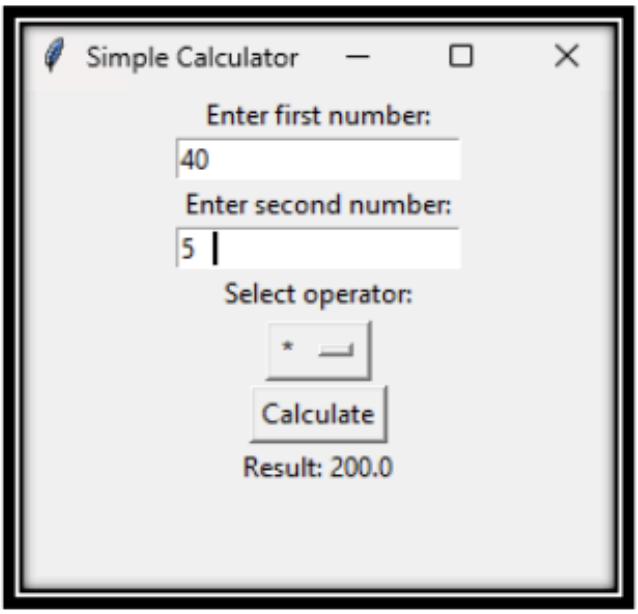
ALGORITHM :

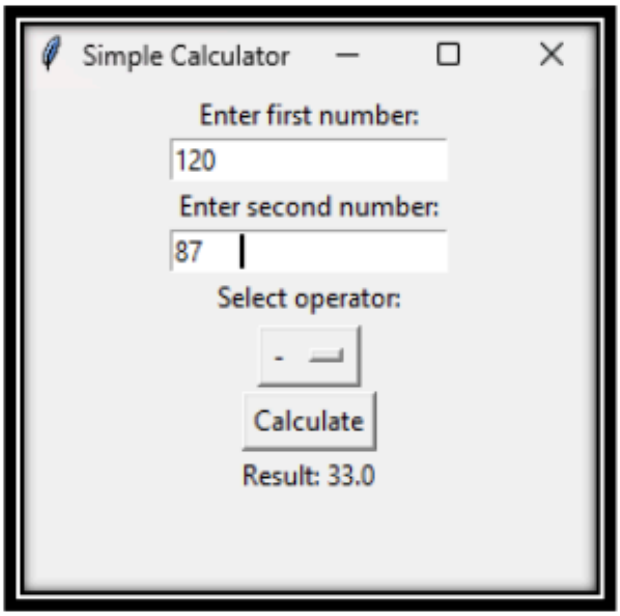
CODE :

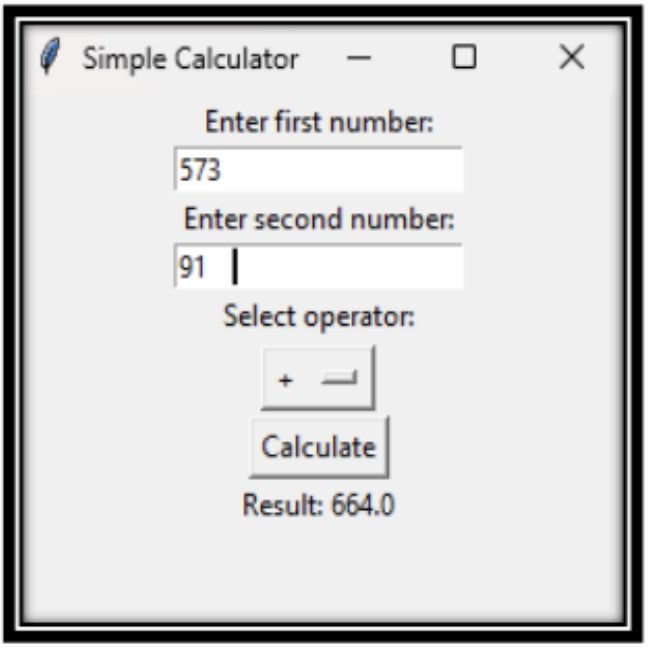
```
import tkinter as tk
from tkinter import messagebox
# Function to perform calculations
def calculate():
    try:
        num1 = float(entry1.get())
        num2 = float(entry2.get())
        operator = operator_var.get()
        if operator == "+":
            result = num1 + num2
        elif operator == "-":
            result = num1 - num2
        elif operator == "*":
            result = num1 * num2
        elif operator == "/":
            if num2 == 0:
                raise ZeroDivisionError
            result = num1 / num2
        else:
            result = "Invalid Operator"
        result_label.config(text=f"Result: {result}")
    except ValueError:
        messagebox.showerror("Input Error", "Please enter valid numbers.")
    except ZeroDivisionError:
        messagebox.showerror("Math Error", "Cannot divide by zero.")
```

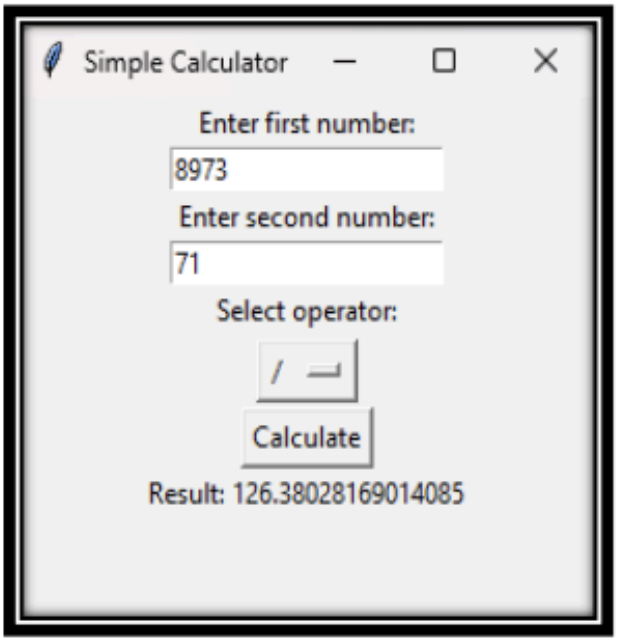
```
# Create main window
root = tk.Tk()
root.title("Simple Calculator")
root.geometry("300x250")
# Input fields
tk.Label(root, text="Enter first number:").pack()
entry1 = tk.Entry(root)
entry1.pack()
tk.Label(root, text="Enter second number:").pack()
entry2 = tk.Entry(root)
entry2.pack()
# Operator selection
operator_var = tk.StringVar(value="+")
tk.Label(root, text="Select operator:").pack()
operator_menu = tk.OptionMenu(root, operator_var, "+", "-", "*", "/")
operator_menu.pack()
# Calculate button
calc_button = tk.Button(root, text="Calculate", command=calculate)
calc_button.pack()
# Result display
result_label = tk.Label(root, text="Result: ")
result_label.pack()
# Run the application
root.mainloop()
```

OUTPUT :


Simple Calculator
Enter first number:
40
Enter second number:
5
Select operator:
*
Calculate
Result: 200.0


Simple Calculator
Enter first number:
120
Enter second number:
87
Select operator:
-
Calculate
Result: 33.0


Simple Calculator
Enter first number:
573
Enter second number:
91
Select operator:
+
Calculate
Result: 664.0


Simple Calculator
Enter first number:
8973
Enter second number:
71
Select operator:
/
Calculate
Result: 126.38028169014085

EXPERIMENT - 12

AIM :

ALGORITHM :

CODE :

```
import requests
import sqlite3
from bs4 import BeautifulSoup
# Database setup
def create_database():
    conn = sqlite3.connect('quotes.db')
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS quotes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            quote TEXT,
            author TEXT
        )
    ''')
    conn.commit()
    conn.close()

# Scrape quotes from the website
def scrape_quotes():
    url = "http://quotes.toscrape.com"
    response = requests.get(url)
    if response.status_code != 200:
        print("Failed to retrieve the page.")
        return []
    soup = BeautifulSoup(response.text, "html.parser")
```

```
    quotes_list = []
    for quote_block in soup.find_all("div", class_="quote"):
        quote_text = quote_block.find("span", class_="text").text
        author = quote_block.find("small", class_="author").text
        quotes_list.append((quote_text, author))
    return quotes_list

# Store data in SQLite database
def store_data(quotes):
    conn = sqlite3.connect('quotes.db')
    cursor = conn.cursor()
    cursor.executemany("INSERT INTO quotes (quote, author) VALUES (?, ?)", quotes)
    conn.commit()
    conn.close()
    print("Data stored in database successfully.")

# Run the scraper and store data
if __name__ == "__main__":
    create_database()
    quotes = scrape_quotes()
    if quotes:
        store_data(quotes)
        print("Scraping and database storage completed!")
```

For showing store data in file quotes.db

```
import pandas as pd

conn = sqlite3.connect("quotes.db")

df = pd.read_sql_query("SELECT * FROM quotes;", conn)

conn.close()


# Display the data

df.head()
```

OUTPUT :

index	id	quote	author
0	1	"The world as we have created it is a process of our thinking. It cannot be changed without changing our thinking."	Albert Einstein
1	2	"It is our choices, Harry, that show what we truly are, far more than our abilities."	J.K. Rowling
2	3	"There are only two ways to live your life. One is as though nothing is a miracle. The other is as though everything is a miracle."	Albert Einstein
3	4	"The person, be it gentleman or lady, who has not pleasure in a good novel, must be intolerably stupid."	Jane Austen
4	5	"Imperfection is beauty, madness is genius and it's better to be absolutely ridiculous than absolutely boring."	Marilyn Monroe

EXPERIMENT - 13

AIM :

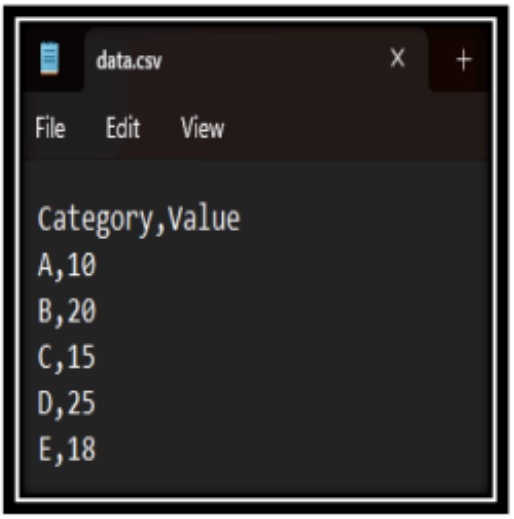
ALGORITHM :

CODE :

```
import pandas as pd

import matplotlib.pyplot as plt

# Read data from a CSV file

df = pd.read_csv("data.csv")

# Create a bar chart

plt.figure(figsize=(8, 5))

plt.bar(df["Category"], df["Value"], color="skyblue")

# Add labels and title

plt.xlabel("Category")

plt.ylabel("Value")

plt.title("Data Visualization Example")

# Show the visualization

plt.show()
```

OUTPUT :

data.csv



EXPERIMENT - 14

AIM :

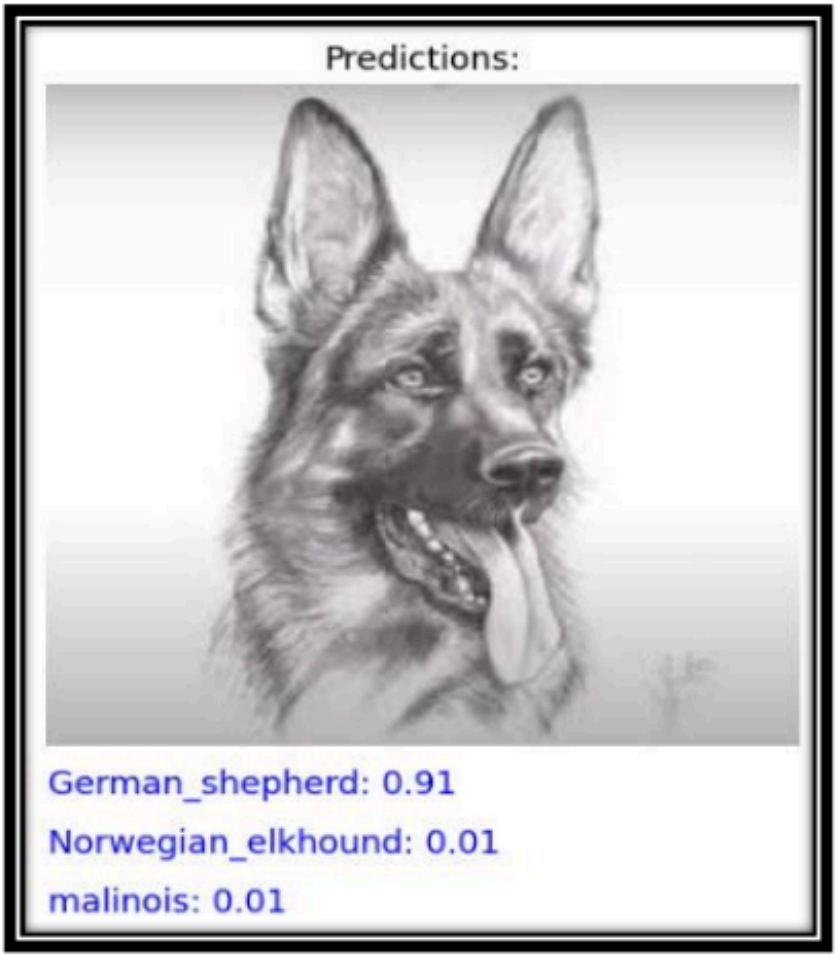
ALGORITHM :

CODE :

```
import tensorflow as tf

import numpy as np

from tensorflow.keras.applications.mobilenet_v2 import MobileNetV2,
preprocess_input, decode_predictions

from tensorflow.keras.preprocessing import image

import matplotlib.pyplot as plt


# Load the pre-trained MobileNetV2 model

model = MobileNetV2(weights="imagenet")


# Load and preprocess an image

def classify_image(img_path):

    img = image.load_img(img_path, target_size=(224, 224))  # Resize to model's
input size

    img_array = image.img_to_array(img)

    img_array = np.expand_dims(img_array, axis=0)

    img_array = preprocess_input(img_array)  # Normalize the image


    # Make predictions

    predictions = model.predict(img_array)

    decoded_predictions = decode_predictions(predictions, top=3)[0]  # Get top
3 predictions


    # Display image and predictions
```

```
    plt.imshow(img)
    plt.axis("off")
    plt.title("Predictions:")
    for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
        plt.text(0, 240 + i * 20, f"{label}: {score:.2f}", fontsize=12, color="blue")
    plt.show()

# Example usage (change 'image.jpg' to your actual image path)
classify_image("images.jpg")
```

OUTPUT :



EXPERIMENT - 15

AIM :

ALGORITHM :

CODE :

Server.py

```
import socket
def start_server():
    host = '127.0.0.1'  # Localhost
    port = 12345        # Port to listen on
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind((host, port))
    server_socket.listen(1)
    print(f"Server started on {host}:{port}")
    while True:
        client_socket, client_address = server_socket.accept()
        print(f"Connection from {client_address}")
        message = "Hello, client! You are connected."
        client_socket.send(message.encode())
        client_socket.close()
if __name__ == "__main__":
    start_server()
```

client.py

```
import socket
import socket
def connect_to_server():
    host = '127.0.0.1'  # Server address
    port = 12345        # Server port
```

```
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client_socket.connect((host, port))

    data = client_socket.recv(1024).decode()
    print(f"Server says: {data}")

    client_socket.close()

if __name__ == "__main__":
    connect_to_server()
```

OUTPUT :

```
C:\Users\arya>cd C:/Users/arya/AppData/Local/Programs/Python/Python38-32

C:\Users\arya\AppData\Local\Programs\Python\Python38-32>python server.py
Server started on 127.0.0.1:12345
Connection from ('127.0.0.1', 50336)
```

```
C:\Users\arya>cd C:/Users/arya/AppData/Local/Programs/Python/Python38-32

C:\Users\arya\AppData\Local\Programs\Python\Python38-32>python client.py
Server says: Hello, client! You are connected.

C:\Users\arya\AppData\Local\Programs\Python\Python38-32>
```